# CHAPTER 3
# METHODS AND PROCEDURES OF THE STUDY

This chapter describes the research methods, research design, locale, respondents, instruments, data gathering procedure, analysis tools, statistical treatment, proof of concept, and software development model for LogiCade. LogiCade is an adaptive gamified learning application for Computer Programming 1 (CP1). It combines curriculum-bound HTML5 Canvas activities, verified challenge generation, learner progression, and pseudonymous gameplay records. Formal learning assessments, gameplay performance, software evaluation, and exploratory machine learning are treated as separate sources of evidence.

## 3.1 Methods of Research Used

The study combines developmental research and descriptive-evaluative research, which are identified in the available LogiCade manuscripts. Developmental research addresses the design, implementation, verification, and refinement of the learning application. It covers the learner interface, curriculum-to-stage mapping, activity packages, challenge verifier, progression rules, data synchronization, and administrator functions. The developmental work produces an operational prototype and documented implementation evidence.

Descriptive-evaluative research addresses the characteristics of system use and the evaluations provided by participating learners and qualified evaluators. Gameplay records describe interaction with the application, while questionnaire responses describe perceptions of the implemented system. These observations do not, by themselves, demonstrate learning effectiveness or establish that LogiCade caused a change in achievement.

The learning-outcome evaluation will follow [insert approved evaluation design]. Matched pretest and posttest records will support analysis of within-participant changes where the approved protocol requires both assessments. Learning analytics will describe attempts, scored submissions, hints, timing, completion, and difficulty transitions. The machine-learning work will examine whether pre-decision gameplay evidence can

classify an expert-defined recommendation for the next stage. No predictive result is assumed in advance.

### 3.2 Research Design

The approved research design is [insert approved research design]. The available LogiCade manuscripts describe different designs: the manuscript titled N.THESIS describes a pretest-posttest comparison with a control group, whereas the revised methodology describes a stratified single-group pretest-posttest design. The number of groups, assignment procedure, and comparison condition will therefore be specified as [insert approved group structure and assignment procedure].

The implementation period will be [insert study duration], conducted during [insert academic term and dates]. The nine curriculum weeks are content regions in the application; their existence does not establish that each region will be delivered over one calendar week. Session frequency, session length, required exposure, attendance procedures, and completion criteria will be [insert approved intervention schedule and completion criteria].

For a matched evaluation, the participant identifier will connect baseline assessment, system exposure, and posttest records. Pretest and posttest scores will use comparable content coverage and a documented scoring scale. The unit of analysis for learning gain is the participant. The unit for operational gameplay description may be a stage run, scored submission, or session, with its denominator stated. The unit for difficulty-model development is one participant at one next-stage decision, using only information observed before that decision.

Research strata, if retained in the approved design, will be Knowledgeable, Moderate, and Minimal. They describe baseline groups for sampling or analysis and are excluded from the live difficulty rule. Their score boundaries and allocation are [insert approved stratum cutoffs and allocation]. Differences across strata will be interpreted with attention to unequal exposure, missing records, baseline differences, and regression to the mean. A single-group design cannot isolate the application effect from instruction, practice, maturation, or other concurrent influences.

### 3.3 Locale of the Study

The study will be conducted at [insert loc]. The participating institution, campus, department, and class setting are [insert institution campus and department]. The learning environment will provide access to CP1 instruction and the devices and connectivity required by the approved study procedure. Details of classroom or laboratory access, supervision, and available equipment are [insert study facilities and access arrangements].

The development environment verified for this project is a Windows laptop with a 13th-generation Intel Core i5-13420H processor, 16 GB installed memory, and an NVIDIA GeForce RTX 4050 Laptop GPU with approximately 6 GB dedicated graphics memory. This is development hardware and does not establish the study venue or a deployed server specification. The final hosting arrangement and minimum learner-device requirements are [insert approved hosting and learner-device specifications].

### 3.4 Respondents of the Study

The respondents will be [insert student affiliation], with a total number of [insert total respondents]. Their year level, degree program, and CP1 enrollment status will be [insert eligible year level program and course enrollment]. Participation will be voluntary and governed by the approved consent procedure.

The sampling technique will be [insert approved sampling technique]. The accessible population, recruitment frame, sample-size justification, and recruitment dates are [insert accessible population sampling frame sample-size basis and recruitment dates]. A plan to retain 100 account records or serve a limited number of players daily is an operational plan and will not substitute for an approved research sample or evidence of actual recruitment.

Eligibility will include enrollment in the relevant CP1 learning context, willingness to participate, ability to access the application under the study arrangements, and completion of [insert required consent or assent process]. Additional inclusion criteria and exclusions are [insert approved inclusion and exclusion criteria]. Withdrawal, attrition, and replacement procedures are [insert approved withdrawal and replacement

policy]. Withdrawn participants will not be treated as if a replacement were the same paired observation.

**Table 1 Proposed respondent and evaluator allocation**

| Group | Basis | Number |
|---|---|---|
| Student participants | [insert student affiliation] | [insert total respondents] |
| Knowledgeable stratum if used | [insert approved pretest range] | [insert allocation] |
| Moderate stratum if used | [insert approved pretest range] | [insert allocation] |
| Minimal stratum if used | [insert approved pretest range] | [insert allocation] |
| Programming instructors | [insert qualifications and selection basis] | [insert evaluator count] |
| Software or IT evaluators | [insert qualifications and selection basis] | [insert evaluator count] |

Faculty and software evaluators will assess only the areas appropriate to their expertise and assigned tasks. Their evaluations will be reported separately from learner achievement scores. Recruitment reporting will distinguish individuals invited, consenting, enrolled, completing the required exposure, and providing matched assessment data.

### 3.5 Research Instruments

#### 3.5.1 Pretest and posttest

The proposed pretest and posttest will measure the selected CP1 competencies independently of game rewards or activity accuracy. The test blueprint, approved item bank, number of items, scoring scale, administration time, and content-validation evidence are [insert approved assessment blueprint item count scoring scale administration time and validation evidence]. Comparable assessments will cover the same intended outcomes; the policy on repeated items will be [insert approved equivalent-form or repeated-item policy].

The database provides assessment records with pretest, posttest, and other phases. An instructor-controlled examination endpoint also exists, but it requires a separately approved private item bank. That examination is recorded under the other phase and does not automatically constitute the research pretest or posttest. Neither database support nor an endpoint demonstrates that a validated instrument has already been administered.

### 3.5.2 Background and software evaluation questionnaires

The methodology draft proposes a baseline background questionnaire and a post-use questionnaire concerning engagement, motivation, satisfaction, and the learning experience. The approved statements, administration procedure, construct definitions, and validation records are [insert approved questionnaires and validation records]. Where the proposed five-point agreement format is retained, responses are coded from 1 for Strongly Disagree to 5 for Strongly Agree. Item-specific response anchors and composite scoring will follow [insert approved questionnaire scoring rules].

The draft software evaluation selects functional suitability, usability, reliability, performance efficiency, security, and maintainability. These are a selected subset of the ISO/IEC 25010:2011 product quality model. ISO identifies that edition as withdrawn and replaced by newer standards; therefore, the study will specify [insert approved ISO standard edition and selected characteristics] before finalizing the instrument. A questionnaire based on selected characteristics will not be represented as certification of full standards compliance (ISO/IEC, 2011, 2023).

### 3.5.3 Gameplay event and telemetry records

The instrumented application stores pseudonymous participant, stage, attempt, and event identifiers; curriculum week and stage; assigned and served difficulty; attempt status; scored submissions; correct submissions; hints; question and activity retries; timing; and challenge provenance. The current research views expose attempt-level measures, stage outcomes, topic evidence, and learner sessions. Historical uninstrumented values remain missing rather than being reconstructed as zero.

**Table 2 Operational definitions of recorded gameplay signals**

| Signal | Operational definition and interpretation |
|---|---|
| Mistake count | Incorrect scored submissions. One failed evaluated Run or Check counts once even when several internal test cases fail. |
| Hint usage | Hints actually revealed during the attempt. Availability and affordability must be known before zero hints is interpreted as independent performance. |
| Completion time | Foreground active play time on a successful run. Host pauses and the initial game guide are excluded by the instrumentation. |
| Attempt count | Distinct stage runs, retaining completed, failed, abandoned, restarted, and incomplete outcomes separately. |
| Accuracy | Correct scored submissions divided by scored submissions. Native game accuracy and its original basis are retained separately. |
| First-try success | Completion on the first evaluated run. An exit before evaluation is not automatically a scored failure; the orientation video is excluded. |
| Failure rate | Failed runs divided by completed plus failed runs. The ML window uses the last five comparable runs and requires at least three evaluated runs. |
| Retry count | Additional submissions to the same task or activity within a run, distinct from a level restart or a completed full-level replay. |
| Topic mastery evidence | The live reporting view aggregates first-evaluated submission correctness by objective. The ML builder instead uses first-try success across distinct comparable tasks; these measures must retain different names. |
| Response time | Instrumented prompt-to-submission active time. Attempt reports expose a mean; the ML workflow uses the median of first measured responses per task when events are available. |
| Session duration | Elapsed time of an authenticated browser-tab session through the saved checkpoint; different tabs create separate sessions. |
| Inactivity time | The session tracker marks visible time more than 60 seconds after an observed interaction as inactivity. Hidden time is separate. Lack of input may include attentive reading and is not proof of disengagement. |

Session snapshots are scheduled every 30 seconds and at relevant visibility or account changes. Abrupt closure can lose time since the latest checkpoint. The supplied difficulty rubric refers to pause or hidden time as inactivity, whereas the implemented

session tracker distinguishes these intervals. The finalized research data dictionary will use [insert approved inactivity definition] and preserve the available components. Interaction tracking does not capture the content of individual keystrokes.

### 3.5.4 Student performance scoring rubric

The supplied Student Performance Assessment Rubric proposes a 100-point score: 70 points for functional correctness, 20 for distinct concept-application checks, and 10 for required deliverables. The proposed score is 70(P/T) + 20(C/K) + 10(D/R), where P/T is the fraction of approved tests passed, C/K the fraction of nonduplicative concept checks satisfied, and D/R the fraction of required artifacts completed. Denominators and any modality-specific alternative must be approved before scoring.

This rubric remains a proposed instrument, not an established automatic grading feature of every current game. Its ten-level Week 1 aggregation conflicts with the implemented two-stage orientation region. The revised level coverage, weights, criteria, assessor procedure, and weekly aggregation rule are [insert approved scoring rubric and weekly aggregation rule]. The orientation video is unscored. Incomplete participation will include assessed-stage counts and missingness rather than an invented completed weekly grade.

### 3.5.5 Adaptive difficulty labeling rubric

The supplied difficulty rubric treats signal bands as review cues. They are not automatically averaged or converted into the final label. Independent instructors will review pseudonymous evidence available before each decision, with automated recommendations and later outcomes concealed. Reviewer identities, evidence notes, disagreements, adjudication, and rubric version will be retained. Reviewer recruitment and adjudication arrangements are [insert approved reviewer and adjudication procedure].

**Table 3 Proposed expert labels for the next stage**

| Label | Evidence described in the supplied rubric |
|---|---|
| Needs Support | Across at least two comparable tasks, repeated prerequisite misconception or need for stepwise guidance. One slow response or failure is insufficient. |

| Label | Evidence described in the supplied rubric |
|---|---|
| Standard | Across at least two comparable tasks, independent work on familiar prerequisites or error correction with a brief prompt. |
| Advanced | Across at least three distinct tasks, independent prerequisite application including a transfer example. |
| Insufficient evidence | Sparse, unevaluated, or unclear relevant history. Leave the training target missing; this is a workflow state, not a fourth prediction class. |

The training scripts normalize the three model labels to needs_support, standard, and advanced. Stage difficulty describes an instructional treatment for the upcoming task and will not be interpreted as a permanent learner label. The minimum evidence, band boundaries, and treatment associated with each category require [insert instructor validation and pilot approval].

### 3.6 Data Gathering Procedure

1. Obtain approval. Secure [insert institutional permission and ethics approval details]. Confirm the research design, sampling procedure, venue, schedule, instruments, consent documents, and participant-data handling plan before recruitment.

2. Prepare curriculum and instruments. Review the CP1 syllabus and stage objectives with [insert subject validators]. Approve the pretest and posttest, questionnaires, difficulty rubric, and any performance scoring rubric. Document pilot procedures and revisions as [insert pilot details].

3. Prepare the application. Record the source revision, curriculum version, schema migrations, model tag and digest, software dependencies, and hosting configuration. Check registration, account uniqueness, access controls, progression, reward idempotency, challenge verification, and synchronization using non-study records.

4. Recruit and obtain consent. Invite eligible [insert student affiliation] using the approved recruitment and sampling procedure. Assign pseudonymous research identifiers and keep identifying account information separately restricted.

5. Collect baseline evidence. Administer the approved background questionnaire and pretest before the required system exposure. Apply stratum cutoffs only if the approved sampling and analysis plan requires them.

6. Conduct system exposure. Deliver [insert approved session schedule and exposure]. Learners register, log in using student numbers, complete initial setup when required, select CP1, enter the world map, and progress through available stages. Observe support and accessibility arrangements without replacing the learner response with researcher answers.

7. Capture and synchronize gameplay. Store instrumented attempts and events locally before sending the outbox to the authenticated server. Confirm parent records before child records and acknowledge successful server storage. Reconcile pending uploads, duplicated identifiers, missing attempt sequences, incomplete runs, and challenge revisions before analysis.

8. Collect endpoint evaluations. Administer the approved posttest after [insert exposure completion criterion]. Obtain the post-use learner questionnaire and role-appropriate software evaluations. Record missing responses, withdrawals, and deviations with reasons where available.

9. Prepare research data. Export permitted pseudonymous tables. Match assessments by participant, verify score scales and timestamps, preserve original status and accuracy basis, exclude duplicate event identities, and retain a missingness report. Limit each ML row to evidence preceding its decision.

10. Label and analyze. Apply the approved expert-label workflow, calculate descriptive and paired statistics, and run the exploratory ML workflow when its data requirements are met. Report the actual number of participants and decisions used in every analysis.

### 3.6.1 Consent and data management

Consent procedures will state the purpose of the study, activities, records collected, access arrangements, and withdrawal options. For participants requiring guardian consent or assent, the applicable process is [insert consent and assent

requirements]. Approval identifiers, data custodian, access roles, retention period, backup location, and disposal procedure are [insert ethics approval custodian access retention backup and disposal details].

The system separates private account credentials from pseudonymous gameplay data, hashes participant passwords, and routes privileged writes through authenticated server operations. Research exports use pseudonymous identifiers by default. Hosting location and external data flows will be documented in the approved deployment plan. Local LLM inference does not mean that no data will leave the laptop if the application or database is later deployed to cloud infrastructure.

### 3.7 Data Analysis Tools

The available implementation uses Python with pandas and NumPy for data preparation; scikit-learn for the exploratory multinomial logistic regression and clustering scripts; PostgreSQL/Supabase for storage and aggregate views; and the administrator panel for inspection and permitted exports. The export implementation includes CSV, Excel, PDF, and Word outputs, although a fully configurable research report builder is not assumed. The inferential statistical package and final versions are [insert approved statistical software and version].

#### 3.7.1 Multinomial logistic regression workflow

The project contains a synthetic-data generator, a real-export feature builder, and an offline trainer for L2-regularized multinomial logistic regression (MLR). The classifier estimates probabilities for the three expert-defined categories from multiple signals jointly. The target is an adjudicated recommendation, not a pretest stratum, a final grade, or the output of the existing rule-based picker. Multinomial logistic regression supports class-probability estimation through a softmax model (scikit-learn developers, n.d.).

Each row contains one next-stage decision. The raw-feature workflow retains relevant numeric values, opportunity denominators, missingness, objective identity, and prior difficulty. A second workflow encodes the interpretation bands. Session duration and inactivity remain contextual fields rather than predictors. Hints are withheld as a

model feature when hint-opportunity data are absent; measured hints remain available for reporting. Response-time medians require instrumented event exports.

The trainer uses a participant-separated holdout with a target fraction of 25% and seed 2026. It requires all three classes in training and evaluation, and searches group splits for class coverage. The code guard of four participants and twelve labeled decisions is only an execution minimum, not evidence of an adequate research sample. All imputation, scaling, encoding, and matched-time baselines are fitted on training participants before holdout evaluation. The final research artifact is refitted on eligible labeled rows after evaluation and is not deployed automatically.

The synthetic generator defaults to 600 decisions from 120 fictional participants. It generates feature patterns and labels together and marks their origin. Such data can check the software pipeline, but model scores from them do not establish predictive validity for students. Synthetic data will not be merged with participant records or reported as research respondents. Production use of a trained model requires [insert real-data validation acceptance criteria and deployment approval].

### 3.7.2 Exploratory learner profiling

An optional K-Means script reads student summaries and uses mean first-attempt score, levels played, total errors, total recorded time, total hints, and overall repeat rate. It standardizes features and compares candidate cluster counts using silhouette scores where the sample allows. The current script fills missing feature values with zero and has limited small-sample safeguards. Consequently, any use in the study will require [insert approved missing-data exposure-normalization and cluster-validation procedure]. No clusters or learner-profile results are assumed, and clusters will not be forced to match the research strata.

## 3.8 Statistical Treatment of the Data

The available revised methodology identifies frequency, percentage, mean, standard deviation, median, paired assessment differences, a paired t-test or conditional nonparametric alternative, and effect size. The final analysis will use only measurements

supported by the collected records and the approved research design. Every result will identify its analytical unit, denominator, exclusions, and missing values.

### 3.8.1 Descriptive statistics and questionnaire responses

For categorical records, percentage = 100f/N, where f is the observed category count and N is the valid denominator. For continuous measurements, the arithmetic mean is the sum of observations divided by n, and sample standard deviation is the square root of the summed squared deviations divided by n − 1. Median and interquartile range will accompany skewed timing and attempt distributions. Timing summaries will specify whether they describe completed runs, failed runs, or all instrumented runs.

For five-point questionnaire responses, the frequency-weighted item mean is the sum of response values multiplied by their frequencies, divided by the number of valid responses. Response distributions and missing counts will also be retained. Composite scales, reverse coding, verbal interpretation intervals, reliability procedures, and validation thresholds are [insert approved questionnaire aggregation and reliability plan]. Ordinal agreement responses will not be presented as direct measures of learning achievement.

### 3.8.2 Learning gain and paired comparisons

For participant i, learning gain is posttest score minus pretest score on the same scoring basis. Paired analysis includes only valid matched records and reports the number excluded for missing or incompatible scores. A positive difference indicates a higher observed posttest score; it does not by itself identify the cause of the change.

If the approved design supports a within-participant comparison and assumptions for the difference scores are reasonable, the paired t statistic is the mean difference divided by its standard error, t = mean(d)/(s(d)/sqrt(n)), with n − 1 degrees of freedom. The revised draft specifies a significance level of 0.05. The final prespecified hypotheses and decision rule are [insert approved hypotheses and analysis plan]. A Wilcoxon signed-rank procedure may be used when its paired-data and distributional assumptions fit the measurements; the alternative will not be chosen merely to obtain significance.

The proposed paired effect size is Cohen's dz = mean(d)/s(d), reported with [insert approved confidence-interval procedure]. If the three named strata are analyzed separately as one family of tests, the revised draft proposes Bonferroni adjustment, 0.05/3 = 0.0167. This adjustment will apply only if those subgroup tests are prespecified. A group-comparison or interaction analysis requires the approved group structure and a separately documented model.

### 3.8.3 Gameplay and generation indicators

Stage completion rate is the count of completed eligible runs divided by the declared eligible-run denominator. Failure rate is failed evaluated runs divided by completed plus failed evaluated runs. Submission accuracy is correct scored submissions divided by all scored submissions. Incomplete, abandoned, restarted, and replayed activity will retain distinct categories; unrecorded activity will not be counted as a successful completion.

AI quality indicators will identify generation requests, verification passes, rejections, retries, publication success, fallback use, and recorded latency where available. Rates will state whether the denominator is a request, a candidate proposal, or a consumed pool slot. Retained local job history is bounded and database generation logging is best effort; neither will be called a complete lifetime total without reconciliation. Synchronization analysis will distinguish locally pending data from server-confirmed rows.

### 3.8.4 Machine learning evaluation

The trainer reports classification accuracy, macro F1, recall by class, a confusion matrix, multiclass log loss, and multiclass Brier score on the participant-separated holdout. Accuracy is the proportion of correctly classified rows. Macro F1 averages the class F1 values equally; class recall is the fraction of true members of a class recovered by the model. The confusion matrix will retain the stated class order. Log loss and Brier score describe probability predictions and are not proof of calibration by themselves.

Instructor agreement before adjudication will be evaluated using [insert approved agreement statistic and weighting]. The supplied rubric proposes weighted kappa

alongside a confusion table, but reviewer data are required. Uncertainty intervals, a fair rule-based comparison, calibration diagnostics, any ordinal-model comparison, and further participant-grouped validation are [insert approved model evaluation extensions]. The current single holdout remains exploratory, particularly where classes or participants are sparse.

## 3.9 Proof of Concept

### 3.9.1 Application architecture

The proof of concept is the implemented LogiCade application. React and Vite manage the learner journey; FastAPI provides authenticated account, progression, difficulty, package, synchronization, and administrator operations; shared HTML5 Canvas controllers render the activities; and Supabase/PostgreSQL stores synchronized records. IndexedDB and its outbox provide local persistence for pending gameplay data. Ollama supplies candidate challenge parameters to a separate verification and publication pipeline.

**Table 4 Application layers and verified responsibilities**

| Layer | Responsibilities |
|---|---|
| Learner interface | Loading and landing views, registration and student-number login, initial character selection, subject selection, prologue, world map, stage selection, mission modal, game frame, Oracle guidance, pause and result states. |
| FastAPI application | Authenticated serving, stage-difficulty resolution, private package leases, participant-scoped synchronization, coin operations, content and research administration, and permitted exports. |
| Canvas runtime | Reusable flowcharts, circuit workbenches, connected C-program nodes, hidden-object history cases, factory tasks, and code-controlled adventures. |
| Persistent research records | Account and pseudonymous participant records, enrollment metadata, levels, attempts, events, coins, decisions, assessments, sessions, generation records, and audit records. |
| Background authoring | Bounded LLM parameter proposal, semantic and executable verification, duplicate detection, recoverable publication, revision retention, and retry scheduling. |

A participant owns multiple stage records and attempts; events refer to the appropriate stage and attempt. The private account record links to a pseudonymous student identifier. Assessment and session records link to that identifier without requiring identifying account values in research exports. Schema migrations also define research views and server access controls. Migration-file presence does not establish which migrations have been applied to a particular hosted database.

### 3.9.2 Curriculum and activity organization

The current catalog contains 2,700 canonical challenge slots, derived from nine weeks, ten source levels, three difficulty categories, and ten variations per category. Week 1 exposes only two stages, so 240 historical slots are unavailable to learners. The registry consequently lists 2,460 playable records over 82 stage positions. These are parameterized challenges sharing authored controllers, not 2,700 independent game engines. The earlier approved 90-activity target and the implemented 82-stage sequence require an explicit mapping of [insert approved research activity count and completion mapping].

**Table 5 Current curriculum regions and game presentations**

| Week | Lesson focus | Current presentation |
|---|---|---|
| 1 | Orientation and interface practice; two available stages | Welcome video followed by a guided drag-and-drop flowchart puzzle. |
| 2 | Flowcharting | Circuit-presented executable charts with sequence, input/output, decisions, loops, and synthesis. |
| 3 | Algorithms | Connected terminal nodes for input/output, ordering, state traces, arithmetic, repair, and bounded execution. |
| 4 | Introduction to C, history, characteristics, and classification | Hidden-object archive cases. Each game presents five artifacts with evidence cards and topic-specific checks. |
| 5 | C components, reserved words, preprocessing, and source organization | Factory and source-assembly activities, including production routing and controllers. |

| Week | Lesson focus | Current presentation |
|---|---|---|
| 6 | Creating simple C programs and program structure | Code-controlled programming adventures with approved C-subset tasks and map goals. |
| 7 | Basic C parts and variable declarations | Connected C-program terminal nodes, typed variables, arithmetic, characters, and integer division. |
| 8 | Local and global variables | Scope-aware C circuit workbenches with branching, merging, and bounded feedback behavior. |
| 9 | Review of Weeks 2 to 8 | Stage-specific flowchart, terminal, archive, factory, adventure, and scope review; Stage 10 is a cumulative expedition. |

Each stage binds its pool to a stable learning objective and source lesson. Variation changes inputs, values, questions, layouts, or routing while preserving the assigned objective and difficulty. This prevents a random activity from replacing the required level topic. Week 9 uses an explicit source-level blueprint and a cumulative capstone rather than treating every review stage as the same exercise.

### 3.9.3 Selection packaging and challenge replacement

After stage selection, the server resolves the participant's assigned difficulty and selects an existing verified JSON from the matching week, level, and difficulty folder. Its description supplies the stage details and mission information. Preparing the mission compiles a small ZIP containing the launcher and selected game data, then extracts a private temporary lease. The shared Canvas code and assets remain outside the per-stage archive. Starting opens that lease in the sandboxed game frame.

Completion, Exit, and Repeat release the active extracted copy and consume the compiled revision. The canonical source remains usable while a background replacement is attempted. Repeat can therefore select another existing verified challenge without waiting for model inference. The model proposes bounded parameters for the same objective and difficulty; authored templates construct the executable challenge. Arbitrary

generated JavaScript, HTML, shell commands, and asset URLs are excluded from this native authoring contract.

The verifier checks the binding, constraints, novelty, executable reference behavior, and rejection of an invalid solution. A passing candidate replaces the source, mirrored definition, ZIP, description, and manifest through a recoverable publication process. A single worker serializes generation, attempts a proposal up to three times, and then schedules another attempt after five minutes if needed. Failure preserves the last verified content. The welcome video is excluded from regeneration. Model identity is configuration-dependent; the exact tag, digest, generation settings, and availability for the study will be [insert deployed model tag digest and generation configuration].

Shared sprites and interface audio reference the existing Kenney asset collection. The C execution environment is a bounded teaching interpreter rather than a complete native C compiler. Supported constructs and helpers are restricted, and program execution has budgets. Consequently, successful in-game execution will not be represented as full ISO C conformance or arbitrary-code compilation.

### 3.9.4 Current difficulty selection and proposed MLR integration

The live engine uses rule version cp1-60-v2. It uses the first evaluated completed or failed run of the preceding stage and the number of completed full-level replays after the first clear. Exits, incomplete runs, and restarts before evaluation do not establish a scored zero. Errors are retained in records but do not affect the current threshold table. Week 1 Stage 2 begins at Standard; subsequent region boundaries carry forward eligible preceding-stage evidence. The video provides no scored adaptation evidence.

**Table 6 Implemented next-stage difficulty rule**

| First evaluated accuracy | Completed full-level replays | Next category |
|---|---|---|
| Below 60% | Any | Needs Support |
| 60% to below 80% | Two or more | Needs Support |
| 60% to below 80% | Zero or one | Standard |

| First evaluated accuracy | Completed full-level replays | Next category |
|---|---|---|
| 80% or higher | Two or more | Standard |
| 80% or higher | Zero or one | Advanced |

The server enforces the assignment when serving and preparing a stage, while selected and actual served difficulty are stored separately. A verified start freezes the served decision. The MLR scripts described in Section 3.7 are offline research tools and do not replace this live table. If model-controlled selection is included in the final study, its integration, approved fallback for insufficient evidence, model version, and acceptance evidence are [insert validated MLR integration and approval details].

### 3.9.5 Rewards synchronization and operational verification

New accounts receive an initial balance of 20 coins through an auditable ledger, and an Oracle hint costs 6 coins. First-clear rewards are issued once, with the learner interface showing 10 coins and 50 experience points for a first clear. Coin operations use server authority and idempotent transactions rather than accepting a student-supplied balance. Experience, coins, and gameplay progress are engagement measures and are separate from formal learning-outcome scores.

The local outbox uploads records explicitly; Supabase does not synchronize arbitrary IndexedDB records automatically. Unique event and attempt identities, ownership validation, parent-before-child upload, and server acknowledgments support repeat-safe synchronization. Administrator views expose synchronized information, which can lag behind unsent local records. Full offline downloading of all game packages and distributed multi-worker publication are not established capabilities of this release.

The verification procedure will cover account uniqueness and access, first-clear and hint transactions, restart versus retry versus replay semantics, difficulty boundaries and carryover, objective binding, package preparation and cleanup, rejected model proposals, revision consistency, missing-data recovery, and export reconciliation. Existing project tests and reports provide development evidence, but study-specific

execution dates, selected build, observed defects, and acceptance results are [insert study-build verification and acceptance records].

### 3.10 Software Model

The available LogiCade methodology specifies an Agile iterative development approach. The application is developed through repeated planning, design, implementation, verification, review, and refinement rather than treating the first prototype as the final study build. Iteration records and formal review dates are [insert development iteration and review records]. The evaluated release will be [insert application source revision schema curriculum and instrument versions].

1. Planning. Map CP1 learning outcomes to regions and stage objectives, define the learner and administrator requirements, and resolve the research-design and instrument approvals.

2. Design. Specify the interface journey, schema relationships, shared Canvas controllers, bounded challenge contract, package lifecycle, difficulty authority, and research data dictionary.

3. Implementation. Build the learner and administrator functions, reusable activities, verification and replacement queue, ledger, local outbox, server access controls, and offline ML tooling.

4. Verification. Review curriculum alignment and instruments with qualified evaluators, execute relevant software checks, inspect failure recovery and exports, and document limits of the C interpreter and content generator.

5. Deployment. Freeze the approved build and configuration, document [insert production hosting and backup arrangements], complete operational acceptance, and orient participants using the approved procedure.

6. Review and refinement. Record defects and feedback, evaluate data completeness and model limitations, and version any approved change. Preserve the study-build identity so results from different instruments or algorithms are not combined without explanation.

**References and Source Materials**

ISO/IEC. (2011). ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation — System and software quality models. https://www.iso.org/standard/35733.html

ISO/IEC. (2023). ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation — Product quality model. https://www.iso.org/standard/78176.html

scikit-learn developers. (n.d.). LogisticRegression. scikit-learn documentation. Retrieved November 8, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Atienza, S. S., Berwega, N. H., Revilla, K. C., & Saliwan, J. B. (2027). LogiCade Development of a Gamified Learning Application for Fundamental Programming Courses Among CCS Students Using Machine Learning [Unpublished thesis manuscript; file titled N.THESIS [1] (2).pdf]. The year reproduces the manuscript title page and does not establish publication.

LogiCade project. (n.d.). Chapter 3 Methodology LogiCade Revised Adaptation Rules [Unpublished working methodology].

LogiCade project. (n.d.). Adaptive Difficulty Classification Rubric [Instrument for instructor validation; supplied Word document].

LogiCade project. (n.d.). Student Performance Assessment Rubric [Instrument for instructor validation; supplied Word document].

LogiCade project. (n.d.). SYLLABI [Supplied course document].

LogiCade project. (2026). Application source and research documentation [Local project snapshot reviewed November 8, 2026].

**Implementation evidence**

The implementation descriptions are supported by data/curriculum.json; games/registry.json; scripts/build_canvas_games.py; server/main.py; server/stage_difficulty.py; server/difficulty_engine.py; server/course_assessments.py; server/canvas_replacements.py; client-react/src/App.jsx; supabase/migrations/001 through 017; and the shared games/runtime modules. Model-development procedures are supported by scripts/build_mlr_difficulty_dataset.py, scripts/generate_synthetic_mlr_dataset.py, scripts/train_difficulty_mlr.py, and analysis/profile_clusters.py.